RateLimiter: Fault-Tolerant Microservice (Part-1)

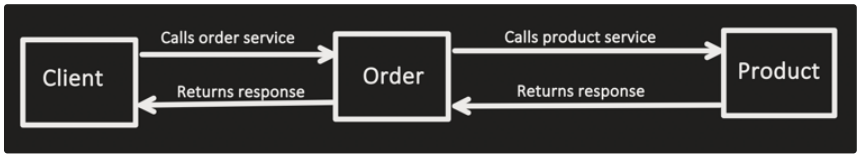
What is Fault-Tolerant Microservice?

- Fault Tolerant Microservice is a service, which continues to work even when downstream system fails.
- Instead of crashing or Cascading failures, it handles the failure gracefully.

What its required?

- As in microservices, different services communicate over the network.
- And its possible that 1 service unavailability could bring down the whole system.

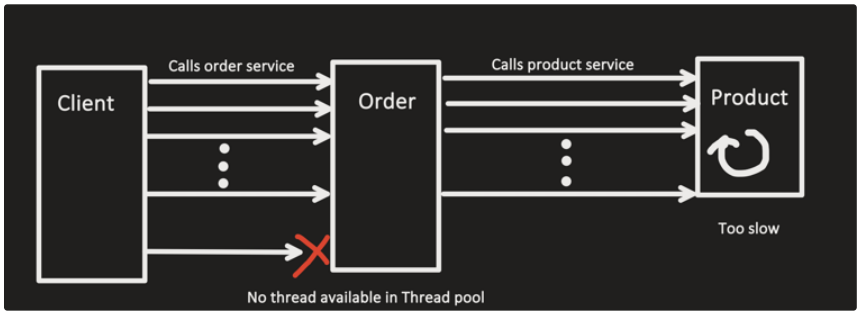
Normal Scenario:



Lets say because of either buggy code or Infrastructure issue "Product" service becomes too slow:

- Say, product service become slow and taking 60 seconds to respond.
- If proper fault tolerant is not applied, then a call from Order service will hang for 60sec.
- Thus, a thread is blocked for that period.

Concept && Coding



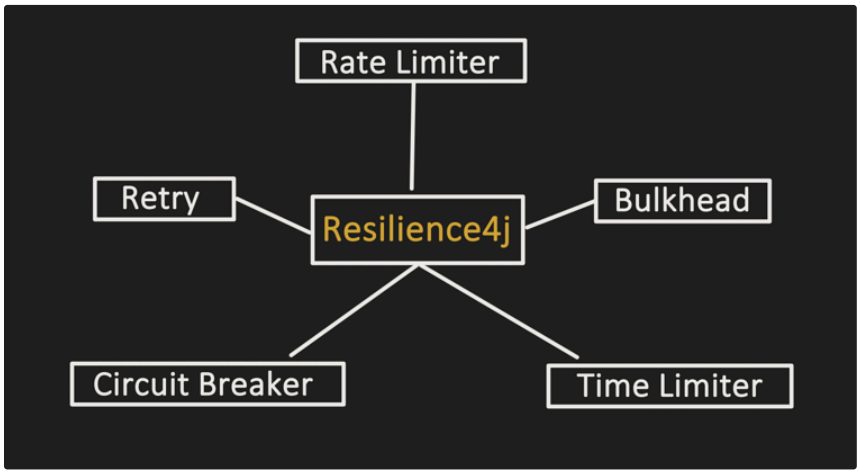
- if sudden burst of request comes from Order to Product service, all threads will now get blocked (waiting for response).
- And because of this, clients of Order service also get blocked.
- Eventually, services will run out of threads and start rejecting the request.

  This is know as CASCADING FAILURE.

means, issue is at Product service, but now propagated to other services too.

To avoid this, we need to build Fault tolerant microservice.

And here comes:



Lets go in depth one by one in recommended logical ordering:

RateLimiter → Bulkhead → TimeLimiter → CircuitBreaker → Retry

Why Ordering required?

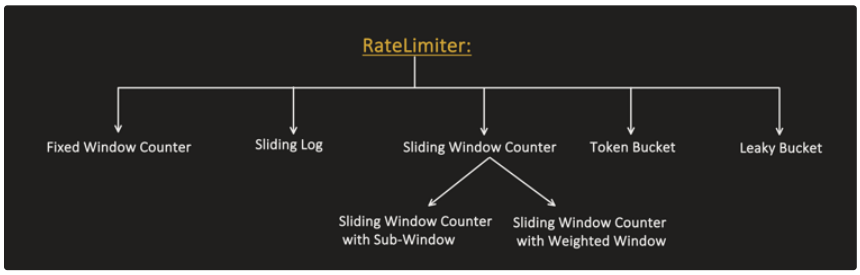
For example: Always apply RateLimiter before Retry, so that retry logic doesn't overwhelm the system with already limited traffic.
Else we end up in retry the traffic, which will get blocked by RateLimiter later.

Concept && Coding

RateLimiter:

It controls the number of requests allowed to a microservice in a given time window.

So in other words, it protect our system from sudden traffic spike (like in DDOS attack).



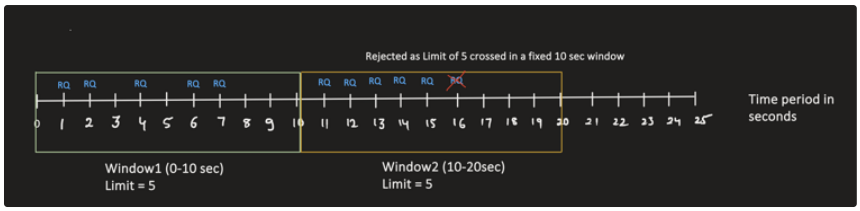
Fixed Window Counter:

- Counts how many request happens in a fixed window.
- If the count crossed the Limit, then reject the request.

For example:

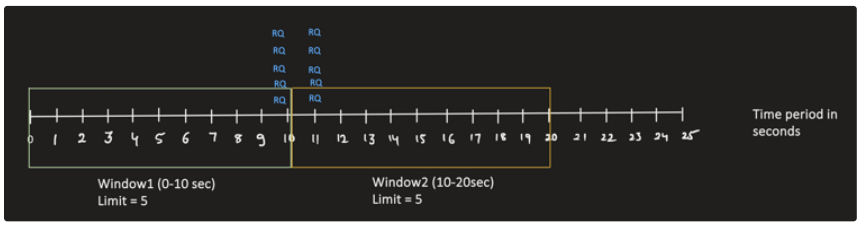
Limit = 5

Fixed Window Size = 10 second



Disadvantage:

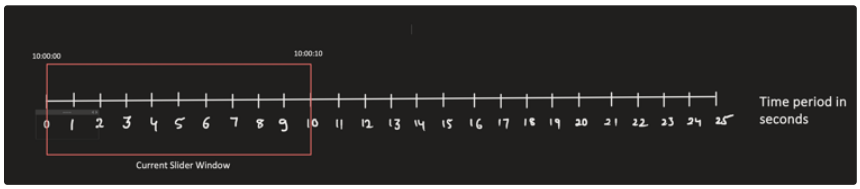
- If traffic comes at the edge (end edge of 1st window and starting edge of 2nd window), then there is a chance of sudden spike.
  like below even though Limit = 5 in 10second, but here 10 requests comes in just 1 second.



<u>Sliding Log:</u>

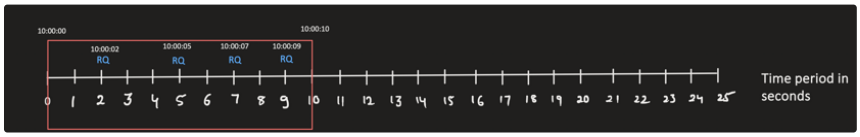
Concept && Coding

- For each accepted request, it stores the exact timestamp.
- For each request comes in, it first check, how many request happened in last X seconds (Window size).
- If no of requests reach the LIMIT, it will reject the request.



Window Size = 10 seconds

Limit = 5



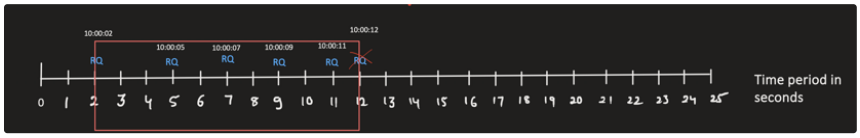
Window Size = 10 seconds

Limit = 5

Now, when window size crossed, slider moves on.
So it consider only those request in count, which are in slider window only.

Window Size = 10 seconds

Limit = 5



Window Size = 10 seconds

Limit = 5

Now, new request coming, but Total no of request in window is already 5. Limit is reached so new RQ will be rejected.

Now, if new request comes and in the current window only 4 requests are there, so new request will be accepted.

Concept && Coding

Disadvantage:

- Need to maintain timeframe info for each accepted request. Which needs more memory.
- Extra overhead of clean up of expired timestamps.

<u>Sliding Window Counter:</u>

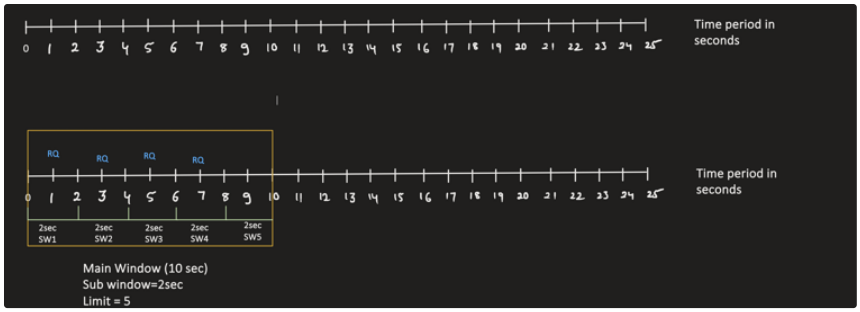
<u>Sliding Window Counter with Sub-Window:</u>

- Main window is divided into equal sized sub-windows.
- Keep Tracks of the number of request in each sub-window.
- To decide if request need to be rejected or not, it consider only active sub-windows.
- Main window slides(by sub-window size) over time.

Window Size = 10 seconds

Sub-Window = 2 seconds

Limit = 5



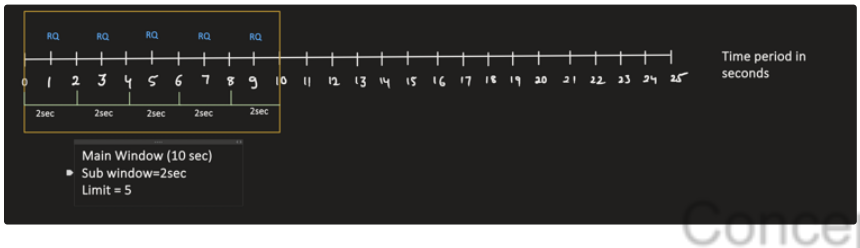
Say for new Request RQ coming at 9th sec time period:

It will sum up the requests in all active Sub-windows:

SW1(0sec-2sec) + SW2(2sec-4sec) + SW3(4sec-6sec) + SW4(6sec-8sec) + SW5 (8sec-10sec)

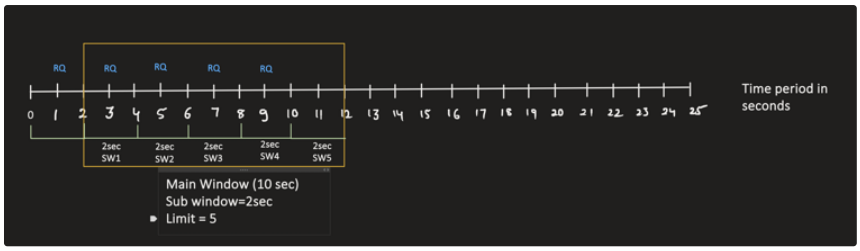
1 + 1 + 1 + 1 + 0 = 4

So its less than Limit 5, so RQ coming at 9th sec time period will be allowed



Concept && Coding

Now when time crosses 10th second, main window has to slide. So it Slide(jump) according to sub-window size.



Say for new Request RQ coming at 11th sec time period:

It will sum up the requests in all active Sub-windows only:

SW1(2sec-4sec) + SW2(4sec-6sec) + SW3(6sec-8sec) + SW4(8sec-10sec) + SW5 (10sec-12sec)

1 + 1 + 1 + 1 + 0 = 4

So its less than Limit 5, so RQ coming at 11th sec time period will be allowed

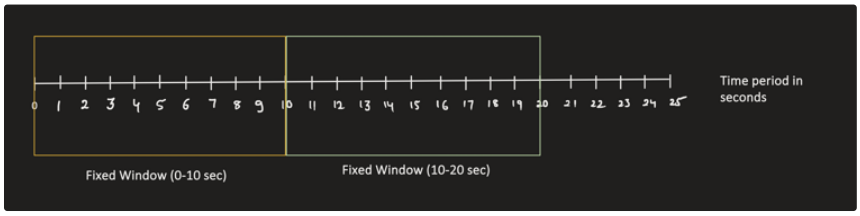
Disadvantage:

- More complex than Fixed window logic.

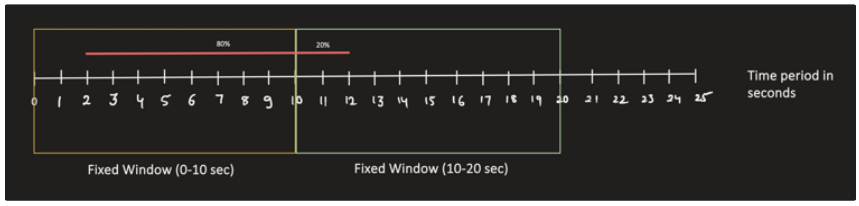
- Need to maintain Sub window info and request count per sub window. Which needs more memory.

<u>Sliding Window Counter with Weighted Window:</u>

- It tries to simplify the logic by removing Sub-window granularity and bringing some approximation.
- It combined Fixed Window + Sliding Log Concept.

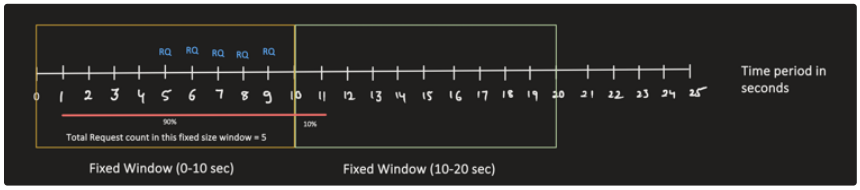


Slider window (equal to size of Fixed Window, say 10 sec) can be between two Fixed Size Window and uses % to determine, how much its in previous window and how much in current window and determine the traffic based on that.



Say, Limit = 5 in a slide window.

Concept && Coding



Now lets say RQ is coming at 11th second of time period.

Now, it uses a formula to compute, how much request from each window it should consider.

Total Requests in Current Fixed Window + 90% of Total Request in previous window (i.e. 5)
(using 90% based on above example shown, it can be different)

Total Requests in Current Fixed Window (10-20sec) = 0

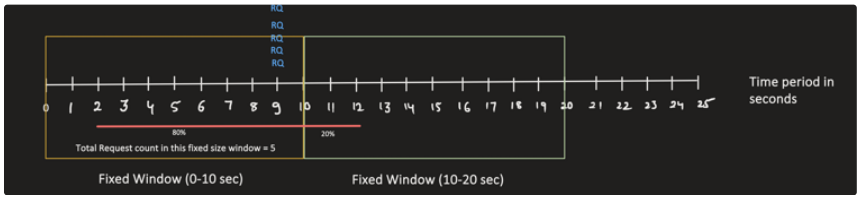
90% of Total Request in previous window (i.e. 5) = 4.5 say we Ceil/round it to ~5

0 + ~5 = 5

Limit is reached of the slider window so Request is Rejected.

Disadvantage:

- It assume uniform distribution of traffic, but that might not be true sometimes.

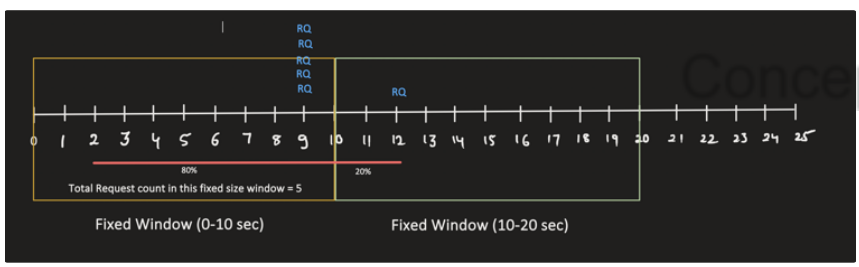


Now, new RQ coming at 12th second

Total Requests in Current Fixed Window + 80% of Total Request in previous window (i.e. 5)
(using 80% based on above example shown, it can be different)

0 + 4 = 4

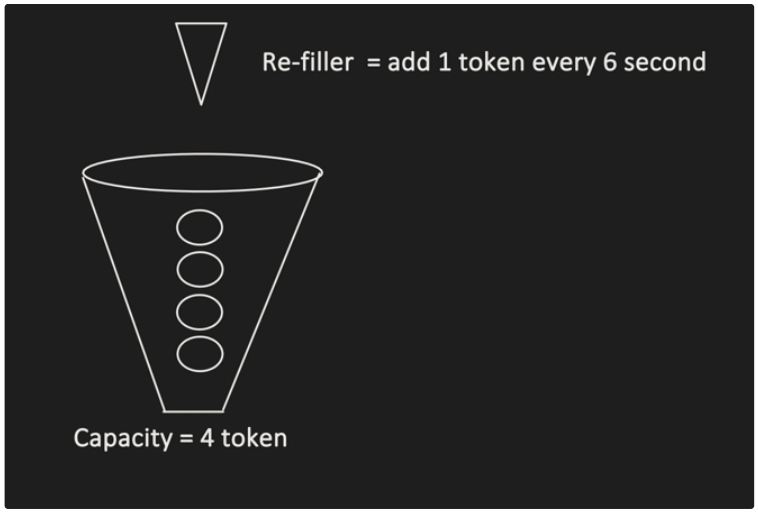
4 is less than Limit 5, so request is permitted

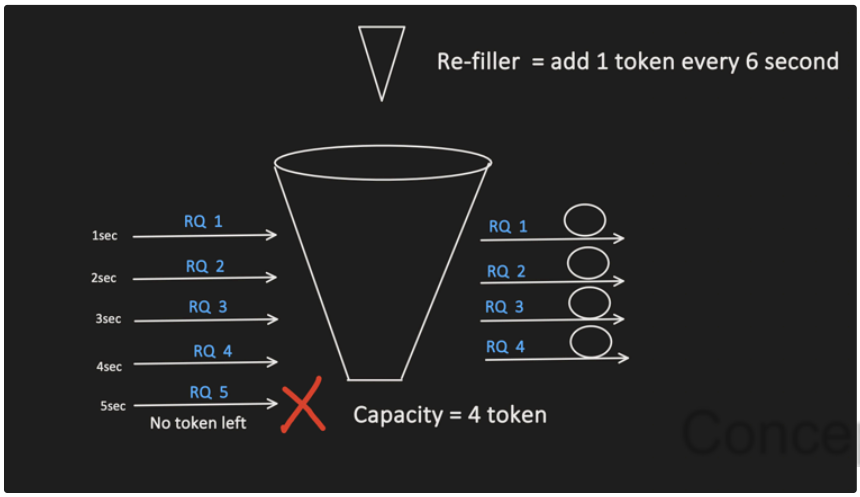


Concept && Coding

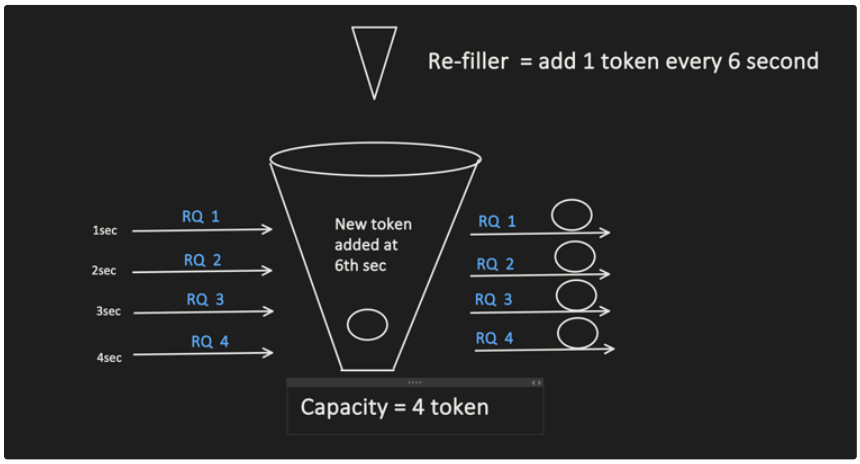
<u>Token Bucket:</u>

- Limited Tokens are placed in a bucket.
- Each request consume one token.
- If there is no token left, request is denied.
- There is a re-filler, which adds the token in bucket at regular time interval.
- If Bucket capacity is full and can not hold more token, token overflow will happen and token will be rejected.





Concept && Coding



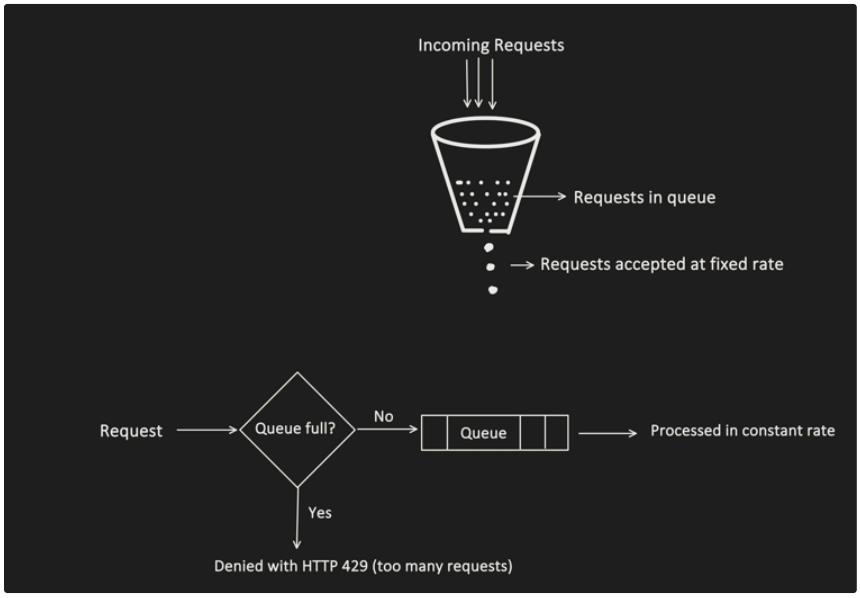
Disadvantage:

- A quick burst is possible if token capacity is not set properly.

Say system remains idle for some time and token keep on accumulating, and when sudden burst comes, it will be allowed as token is present.

Leaky Bucket:

- Requests are Queued and accepted at fixed rate.
- If queue is full, incoming request will overflow.



Disadvantage:

- Latency increases, as request might have to wait in queue for its turn.
- Queue size is a bottle neck. So size should be properly set.

Concept && Coding

Implementation in Spring boot:

Resilience4j by-default applies **token bucket algorithm**

Pom.xml dependency

```
<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-boot3</artifactId>
    <version>2.1.0</version>
</dependency>

```

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    OrderService orderService;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {
        orderService.invokeProductAPI(id);
    }
}

```

```
@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping(value = "/products/{id}")
    String getProductById(@PathVariable("id") String id);
}
```

```

```

```java
@Component
public class OrderService {

    @Autowired
    ProductClient productClient;

   /*
     Internally it uses AOP functionality, that's why
     the method on which this @RateLimiter annotation applies,
     need to be public and Spring managed bean.
   */
    @RateLimiter(name = "productRateLimiter", fallbackMethod = "rateLimitedFallback")
    public void invokeProductAPI(String id) {
        String response = productClient.getProductById(id);
        System.out.println("Response from Product api call is: " + response);
    }

     /*
     Fallback method, return type and parameter
     (additionally only Throwable need to add more)
     should match with the original method, else default
     fallback method provided by RateLimiter framework will get invoked.
  */
    public void rateLimitedFallback(String id, Throwable t) {
        System.out.println("Rate limit exceeded. Try later");
        // throw exception here and handle it gracefully
    }
}

```

Concept && Coding

application.properties

```
server.port=8081
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

# bucket filled with 2 tokens every 10 second,
# if there is no token wait for 1 sec before rejecting the request
resilience4j.ratelimiter.instances.productRateLimiter.limitForPeriod=2
resilience4j.ratelimiter.instances.productRateLimiter.limitRefreshPeriod=10s
resilience4j.ratelimiter.instances.productRateLimiter.timeoutDuration=1s

```

Output:

when frequently tried to hit the API, after frequent 2 calls, tokens finished and for next call, got the Rate limiting message. And after waiting for 10seconds, again token is available and able to hit the api again.

```
Response from Product api call is: fetch the product details with id:1
Response from Product api call is: fetch the product details with id:1
Rate limit exceeded. Try later
Response from Product api call is: fetch the product details with id:1
Response from Product api call is: fetch the product details with id:1
Rate limit exceeded. Try later
Response from Product api call is: fetch the product details with id:1
Response from Product api call is: fetch the product details with id:1
Rate limit exceeded. Try later
```

In similar way, @RateLimiter annotation can be used, if we are using RestTemplate or RestClient. No change in that.

If want to write custom rate limiting logic, we can easily do it, using AOP functionality

Explained AOP in depth in this:

```java
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface CustomRateLimiter {
    int limit();              // max requests
    int windowInSeconds();    // sliding window duration
}

```

Concept && Coding

```java
@Aspect
@Component
public class RateLimiterAspect {

    @Around("@annotation(customRateLimiter)")
    public Object rateLimit(ProceedingJoinPoint pjp,
                            CustomRateLimiter customRateLimiter)
throws Throwable {

        // custom rate limiting logic, if request accepted, invoke the
method
        return pjp.proceed();
    }
}

```

```java
@CustomRateLimiter(limit = 5, windowInSeconds = 60)
public String getProducts() {

    // service call
    return "List of products";
}

```

Next will cover: Bulkhead, TimeLimiter, Circuit breaker and Retry